

RAPiT: RAP inspired Test Suite Generation

David Bruns (20256418)
Department of Computer Science
Technical University of Munich
Munich, Germany
david.bruns@tum.de

Fabian Deigner (20256420)
Department of Computer Science
Technical University of Munich
Heilbronn, Germany
fabian.deigner@tum.de

Andrej Spodarev (20256403)
Department of Computer Science
Technical University of Munich
Munich, Germany
andrej.spodarev@tum.de

Mark Domgoergen (20256408)
Faculty of Computer Science
RWTH Aachen University
Aachen, Germany
mark.domgoergen@rwth-aachen.de

Abstract—Software testing is a fundamental but resource-intensive phase of the development lifecycle, often consuming significant resources. While automated techniques such as Search-Based Software Testing (SBST) have been successful with statically typed languages, they often struggle with the dynamic nature of languages like Python. Conversely, Large Language Models (LLMs) offer vast semantic knowledge but are prone to hallucinations and lack systematic feedback mechanisms.

In this work, we propose RAPiT, a novel approach that bridges the semantic reasoning of LLMs with the systematic exploration of search algorithms. We adapt the Reasoning via Planning (RAP) framework, modeling test generation as a Markov Decision Process (MDP) solved via Monte Carlo Tree Search (MCTS). Unlike standard RAP implementations that rely on LLM self-reflection, our approach utilizes actual execution coverage as the reward signal, providing a ground-truth feedback loop that ensures functional effectiveness. We evaluated RAPiT against Pynguin and a zero-shot LLM baseline on a curated dataset of assignment and open-source library files. The results demonstrate that RAPiT achieves higher median coverage and greater stability, successfully generating tests for complex logic paths where traditional tools fail.

I. INTRODUCTION

Software testing is a fundamental but resource-intensive phase of the software development lifecycle, often consuming up to 50% of development resources [1]. While automated techniques such as Search-Based Software Testing (SBST) have successfully generated high-coverage test suites for statically typed languages like Java, they often struggle with dynamically typed languages such as Python. Traditional tools like Pynguin [2] evolve populations of random sequences but lack a semantic understanding of the code. This frequently results in tests that are syntactically valid but semantically meaningless, particularly when code requires complex input objects or specific program states to penetrate deep logic gates.

Recent advancements in Large Language Models (LLMs) offer a potential solution through their vast semantic knowledge and zero-shot generation capabilities. However, naive LLM-based generation is prone to hallucinations and "writer's block"-generating large volumes of code without verifying execution or coverage. While an LLM can

easily construct complex objects that stymie traditional SBST, it lacks a systematic feedback loop; if a zero-shot prompt misses a branch, the model has no inherent mechanism to correct itself.

In this work, we propose a novel approach that bridges the gap between the semantic reasoning of LLMs and the systematic exploration of search algorithms. We adapt the Reasoning via Planning (RAP) framework [3], treating test generation as a sequential decision-making problem rather than a simple text completion task. By integrating an LLM into a Monte Carlo Tree Search (MCTS) algorithm, we leverage the LLM as a heuristic policy to propose semantically relevant test cases while the MCTS provides the scaffolding to "look ahead," backtrack from failed attempts, and systematically target uncovered code.

Crucially, our approach moves beyond mere self-reflection. We utilize actual execution coverage as the reward signal within the MCTS, providing a ground-truth feedback loop that ensures generated tests are functionally effective rather than just textually coherent.

II. BACKGROUND

A. Reasoning viA Planning

Reasoning viA Planning (RAP) [3] is a Chain-of-Thought (CoT) [4] prompting method that improves the reasoning process of a Large Language Model (LLM). It models the reasoning space of a given problem as a Markov Decision Process (MDP), and then applies Monte Carlo Tree Search (MCTS) to the given problem and its MDP (see Fig. 1). RAP aims to structure the CoT reasoning process by allowing the LLM to explore suboptimal decisions and then backtrack to better ones. It enables the LLM to recover from local optima and ultimately choose the best one.

a) World Model Formulation: The RAP framework reformulates the reasoning process as a planning problem by repurposing the LLM to serve as both a reasoning agent and an

internal world model. Unlike standard autoregressive approaches, RAP models reasoning as a MDP defined by a state space $\mathcal{S}$ and an action space $\mathcal{A}$. At step t , given a state s_t , the agent samples an action $a_t \sim p(a|s_t)$. Crucially, the world model then predicts the next state s_{t+1} via a transition distribution $p(s_{t+1}|s_t, a_t)$. This mechanism allows the LLM to simulate future outcomes and construct a reasoning trace $\tau = (s_0, a_0, s_1, \dots, s_T)$ that is grounded in the predicted evolution of the world state.

b) Planning with Monte Carlo Tree Search: To navigate the vast reasoning space defined by the world model, RAP utilizes MCTS to find high-reward reasoning trajectories. The algorithm iteratively constructs a search tree through four distinct phases:

- 1) **Selection:** The algorithm traverses the current tree from the root, selecting child nodes that maximize the Upper Confidence Bound applied to Trees (UCT), effectively balancing exploration of unvisited paths and exploitation of high-value states.
- 2) **Expansion:** Upon reaching a leaf node, the LLM agent samples d potential actions, and the world model predicts the corresponding next states to expand the tree.
- 3) **Simulation:** To evaluate the potential of a new state, the system performs a look-ahead rollout—simulating a sequence of future steps using the world model—to estimate a cumulative reward.
- 4) **Back-propagation:** The reward obtained from the simulation is propagated backward up the tree to update the value function $Q(s, a)$ for all nodes along the path.

By utilizing the LLM-based world model, and the MCTS algorithm, RAP achieves higher reasoning capabilities than its predecessors which is why it proves to provide a powerful reasoning-based search method.

B. Chain-of-Thought in Test Suite Generation

While LLMs have demonstrated capability in generating code, their application to test suite generation faces unique challenges, particularly

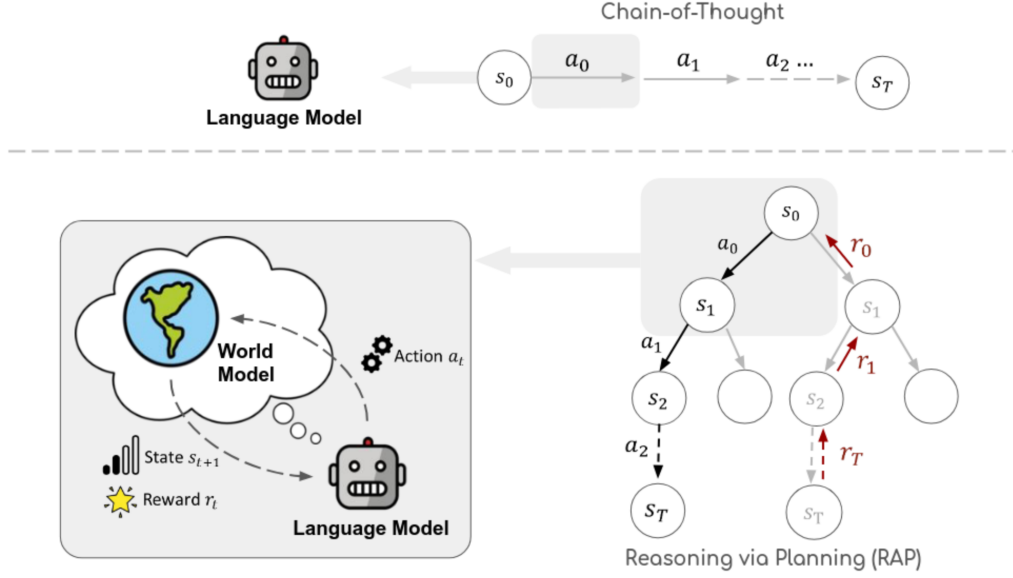


Figure 1. Traditional reasoning approach of **Chain-of-Thought** against the novel **RAP** framework. Image taken from [3]

the **Oracle Problem**, where models fail to predict the correct execution logic of the code under test [5]. To mitigate this, recent research has adopted CoT prompting to decouple reasoning from syntax generation.

a) Reasoning-Based Generation: Standard zero-shot prompting often leads to hallucinated assertions. The *UTGen* framework demonstrates that forcing an LLM to generate an execution trace (rationalization) before the test case significantly aligns generated assertions with the actual code logic, balancing the trade-off between error-revealing inputs and assertion accuracy [5]. Similarly, *HITS* employs CoT to decompose complex methods into slices, reasoning about input conditions for each slice before generating coverage-oriented tests [6].

b) Structured and Multi-Agent Approaches: Beyond free-form reasoning, *Structured Chain-of-Thought (SCoT)* enforces algorithmic structures (e.g., sequence, branch, loop) within the reasoning steps. This structure effectively constrains the LLM to plan input boundaries and assertion logic systematically, reducing "logic drift"

between the setup and verification phases [7].

III. METHODOLOGY

This section details our approach to automating test generation by adapting the Reasoning via Planning (RAP) framework. We model the test generation process as a search problem where an agent incrementally builds a test suite to maximize code coverage.

A. Algorithm

We adapt the RAP framework to the domain of software testing by formalizing the generation of a test suite as a Markov Decision Process (MDP) solved via Monte Carlo Tree Search (MCTS). Unlike standard RAP, which often relies on LLMs for self-evaluation (rewards), our approach integrates a deterministic environment (compiler and coverage tool) to provide ground-truth feedback.

The core components of our algorithm are defined as follows:

- **State (S):** A state represents the current partial test suite and its execution status. Concretely, a state is a tuple containing the

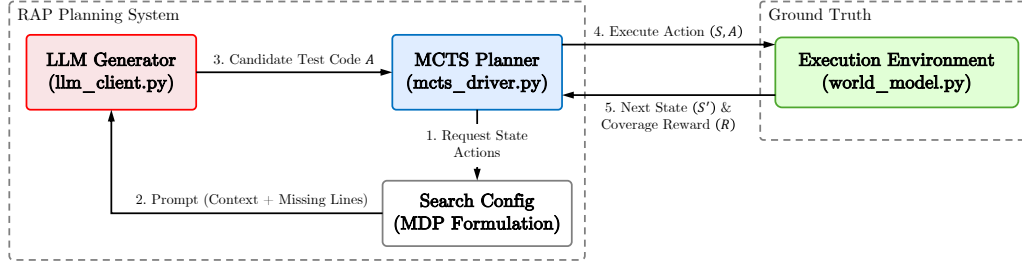


Figure 2. System Architecture of the RAP-based Test Generation. The **MCTS Planner** coordinates the process by (1) requesting actions from the **Search Config**, which (2) prompts the **LLM Generator** with source context and missing lines. The LLM (3) returns candidate test code (A), which the Planner (4) executes in the **Execution Environment** (Ground Truth). Finally, the Environment (5) provides deterministic feedback in the form of the next state (S') and an execution-based coverage reward (R).

sequence of generated test cases, the set of covered line numbers, the set of missing line numbers, and the aggregate coverage percentage. The initial state S_0 contains an empty test suite and 0% coverage.

- **Action (A):** An action is the generation of a single, complete Python test function (e.g., `def test_case_1(): ...`). The LLM serves as the policy $\pi(A|S)$, proposing candidate test functions intended to target specific uncovered lines in the source code.
- **Transition ($T(S, A) \rightarrow S'$):** The transition function is deterministic but computationally expensive. It involves:
 - 1) Appending the new test case A to the existing suite in S .
 - 2) Assembling a temporary executable test file that imports the source module.
 - 3) Executing the suite using `pytest` monitored by `coverage.py`.
 - 4) Parsing the coverage report to update the set of covered/missing lines and calculating the new coverage percentage.
- **Reward (R):** To guide the MCTS, we utilize a dense, execution-based reward signal. The reward for reaching a state S is directly proportional to the code coverage achieved by the test suite at that state: $R(S) = \text{CoveragePercent}(S)/100.0$. This eliminates the hallucination risks associated with LLM-estimated rewards, ensuring the search pri-

oritizes objectively better test suites.

The goal of the search is to find a terminal state S_{term} where coverage is maximized (ideally 100%) or the computational budget (iteration limit) is exhausted.

B. Implementation

Our system is implemented in Python, leveraging the `llm-reasoners` library [8] for the MCTS backbone and `vllm` for high-throughput LLM inference. The complete source code, including the evaluation scripts and dataset, is available publicly.¹

1) *Architecture:* The system architecture, illustrated in Fig. 2, follows a modular design separating the search logic, the generative model, and the execution environment.

- **MCTS Driver:** The entry point that initializes the search. We configure MCTS with an exploration weight (w_{exp}) of 1.0, a depth limit of 8, and a budget of 20 iterations per file.
- **LLM Client:** We utilize Qwen2.5-Coder (Qwen/Qwen2.5-Coder-7B-Instruct) [9] as our underlying model. To ensure low latency during the extensive sampling required by MCTS, we implement a direct `VLLMDirectClient` that loads the model into GPU memory (VRAM). We

¹<https://github.com/m4rk-git/RAP-inspired-test-suite-generation>

use a context window of 16,384 tokens to accommodate large source files and accumulated test suites.

We also experimented with other 7B parameterized models released by Meta-Llama and Mistral but decided on using Qwen because of its superiority for this specific task.

- **World Model:** This component acts as the environment interface. It manages a temporary workspace for each search run, handles import resolution (converting file paths to Python module strings), and robustly handles execution failures (e.g., SyntaxErrors or Timeouts).

2) *Prompting Strategy:* Prompt engineering is critical for guiding the LLM to generate targeted tests rather than generic ones. We implement a dynamic prompting strategy in the `TestGenConfig` class.

Instead of asking the LLM to "write tests" generally, we frame the task as a **"Python Execution Path Solver."** The prompt includes:

- 1) **Numbered Source Code:** The full source code is provided with line numbers (e.g., `10 | if x > 0:`) to allow the LLM to reason about specific locations. We also tried a different approach to improve the attention of the LLM by highlighting the missing lines (e.g. `>>> | if x > 0:`). But as it did not improve the performance, we reverted to the more stable indexing of lines.
- 2) **Target Missing Lines:** Crucially, we dynamically inject the set of currently *missing* lines (derived from the state) into the prompt. The instruction explicitly tells the model: *"Your ONLY goal is to generate exactly ONE pytest test case that forces the Python interpreter to execute one line listed in TARGET_MISSING_LINES"*.
- 3) **Constraints:** To ensure the output is parseable by our world model, we enforce strict constraints: output only valid Python code, no Markdown formatting, and generate exactly one function starting with `def`

`test_`.

We employ a sampling temperature of 0.8 and generate $n = 3$ candidate actions per MCTS step, filtering out candidates that are too short or syntactically invalid before execution.

IV. EVALUATION

To assess the effectiveness of our RAP-based test generation approach, we conducted a comparative evaluation against two distinct baselines: a traditional Search-Based Software Engineering (SBSE) tool and a standard Large Language Model (LLM) prompting strategy. The evaluation focuses on code coverage, stability, and the ability to handle complex logic.

A. Experimental Setup

1) *Benchmarks:* We curated a dataset consisting of two distinct categories of Python programs:

- **Assignment Suite (7 files):** A set of introductory Python programs from our course curriculum. These files contain simple control flow structures (loops, conditionals) and arithmetic operations, serving as a baseline for verifying basic correctness.
- **Complex Suite (10 files):** A selection of files from open-source Python projects and standard libraries (e.g., `copy.py`, `graphlib.py`). These files present significant challenges, including complex object initialization, heavy use of standard library imports, and deep execution paths.

2) *Baselines:* We compared our method against the following:

- **Pynguin (SBSE Baseline):** We utilized Pynguin [2], the state-of-the-art automated test generator for Python. To ensure a fair comparison regarding computational resources, we configured Pynguin with a generous time budget of 530 seconds per file.
- **Baseline LLM (Zero-Shot):** We implemented a direct prompting baseline using the same underlying model as our approach (Qwen/Qwen2.5-Coder-7B-Instruct [9]). The model was prompted with the

full source code and a zero-shot instruction to generate a comprehensive pytest test suite that covers 100% of the code in a single pass. This baseline measures the raw capability of the LLM without the MCTS feedback loop.

3) *Protocol*: For our RAP approach, we configured the MCTS with a budget of 20 iterations and a depth limit of 8. To account for the stochastic nature of all three methods (Pynguin’s evolutionary search and the LLM’s sampling temperature), we performed 5 independent runs for every file and method. The reported results represent the average line coverage achieved across these runs.

B. Results Analysis

1) *Overall Performance Distribution*: Fig. 3 presents the distribution of coverage scores across all test files.

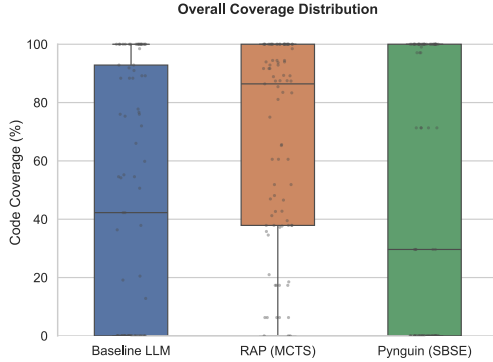


Figure 3. Distribution of Code Coverage across all benchmarks. The RAP approach (Ours) demonstrates higher median coverage and reduced variance compared to the Baseline LLM and Pynguin.

As observed, our RAP approach achieves a significantly higher median coverage compared to Pynguin. While Pynguin performs adequately on simple logic, it struggles with the dynamic typing and complex imports found in the complex suite, often resulting in low coverage outliers. The Baseline LLM shows high variance; without a feedback loop, it frequently generates

syntactically correct but semantically ineffective tests that fail to execute certain branches.

2) *Efficiency Analysis*: To understand the trade-off between execution cost and result quality, we analyzed the relationship between time and coverage, shown in Fig. 4.

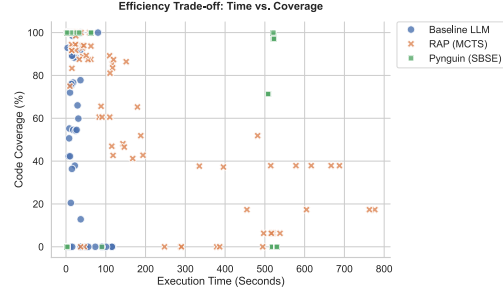


Figure 4. Efficiency Analysis (Coverage vs. Time). Points in the top-left represent ideal performance (fast & high coverage). The distinct cluster of Pynguin points at the bottom-right (Coverage ≈ 0 , Time ≈ 530 s) indicates time-out failures.

The scatter plot reveals three distinct behavior patterns:

- 1) **Baseline LLM (Left Vertical Strip)**: This method is consistently fast (execution time < 100 s) but exhibits high variance in coverage. Since it lacks a feedback loop, the coverage fluctuates purely based on the model’s zero-shot capability for the given file difficulty, without investing more time to fix mistakes.
- 2) **RAP (Linear Downtrend)**: Instead of a fixed cluster, our approach exhibits a diagonal trend. It populates the Top-Left for simple files (fast & high coverage) and extends linearly downwards towards the right as complexity increases. This illustrates a "cost of complexity": harder files require significantly more search time (> 100 s), which naturally correlates with lower final coverage, but the system still manages to explore the search space effectively.
- 3) **Pynguin (Bimodal & Timeouts)**: Pynguin shows a polarized performance. It is either fast and effective (Top-Left) or it consumes

extensive time. Notably, a distinct cluster appears at the bottom-right (530s, 0% coverage). These represent timeout failures where Pynguin exhausted its entire budget without finding a valid entry point, whereas RAP was able to generate at least partial tests for these difficult cases.

3) *Performance on Assignment Tests:* The results on the simple assignment suite are shown in Fig. 5.

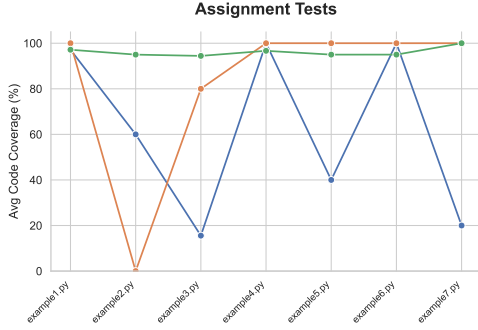


Figure 5. Average Coverage on Simple Assignment Tests (7 Files). Most methods saturate near 100%, though Pynguin fails specifically on `example2.py`. Baseline LLM (blue), RAP with MCTS (green) and Pynguin (orange).

On these simpler tasks, our RAP approach demonstrates remarkable consistency, maintaining coverage above 94% across all runs and never exhibiting catastrophic failure. In contrast, the baselines show significant volatility. Pynguin typically achieves 100% coverage but suffers from binary failures; specifically, it achieved 0% on `example2.py` because the file name contained the string “test” causing a conflict with the tool’s internal logic. The Baseline LLM is even more erratic, fluctuating between perfect 100% coverage on some runs and dropping below 20% on others for the same file, highlighting the unreliability of zero-shot generation without verification.

4) *Performance on Complex Tests:* The most significant differentiation appears in the complex suite, as detailed in Fig. 6.

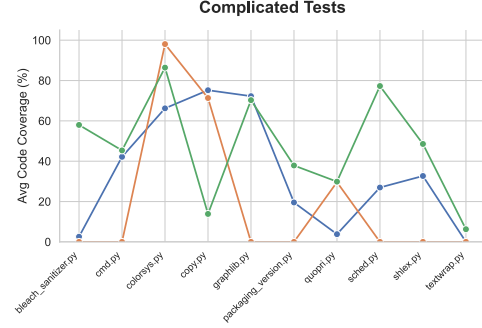


Figure 6. Average Coverage on Complex Real-World Tests (10 Files). RAP outperforms baselines on files with complex imports and state-dependent logic. Baseline LLM (blue), RAP with MCTS (green) and Pynguin (orange).

Here, the limitations of the baselines become apparent. Pynguin often times out (reaching the 530s limit) as it struggles to instantiate the complex objects required for valid inputs. The Baseline LLM suffers from “hallucination”—it generates tests that *look* comprehensive but crash upon execution or miss subtle edge cases due to a lack of feedback. In contrast, RAP incrementally improves by explicitly targeting missing lines in the prompt.

5) *Detailed Numerical Results:* Table I summarizes the exact coverage numbers. Our method achieves the highest average coverage in the majority of complex cases.

V. DISCUSSION

The experimental results demonstrate that the integration of a Large Language Model (LLM) into a Reasoning via Planning (RAP) framework significantly enhances the effectiveness of automated test generation for Python. By combining the semantic depth of LLMs with the systematic exploration of Monte Carlo Tree Search (MCTS), our approach addresses the core limitations of both traditional search-based tools and naive generative models.

A. Semantic Reasoning vs. Random Mutation

Traditional Search-Based Software Testing (SBST) tools, such as Pynguin, rely heavily on

Table I
COMPARISON OF AVERAGE LINE COVERAGE (%) OVER 5 RUNS.

Filename	Plain LLM	Pynguin	RAP (Ours)
<i>Assignment Suite</i>			
example1.py	97.14%	100.00%	97.14%
example2.py	60.00%	0.00%	95.00%
example3.py	15.56%	80.00%	94.44%
example4.py	100.00%	100.00%	96.67%
example5.py	40.00%	100.00%	95.00%
example6.py	100.00%	100.00%	95.00%
example7.py	20.00%	100.00%	100.00%
<i>Complex Suite</i>			
bleach_sanitizer.py	2.57%	0.00%	57.98%
cmd.py	42.18%	0.00%	45.36%
colorsys.py	66.21%	98.06%	86.41%
copy.py	75.20%	71.33%	13.86%
graphlib.py	72.25%	0.00%	70.27%
packaging_version.py	19.55%	0.00%	37.88%
quopri.py	3.83%	29.63%	29.88%
sched.py	26.97%	0.00%	77.27%
shlex.py	32.67%	0.00%	48.55%
textwrap.py	0.00%	0.00%	6.29%
Grand Total	45.54%	45.82%	67.47%

genetic algorithms to mutate input sequences. While effective for simple types, they often fail to generate complex object hierarchies or valid state transitions in dynamically typed languages. As shown in Figure 4, Pynguin frequently timed out or achieved 0% coverage on complex files because it could not "guess" the required input structures.

In contrast, the Qwen2.5-Coder model, acting as the MCTS policy, utilized its pre-trained knowledge of Python libraries and design patterns to construct semantically valid objects. This allowed the search find deep branches that were previously inaccessible to random or evolutionary mutation.

B. The Necessity of the Execution-Based Reward

One of the most critical findings is the performance gap between the Baseline LLM (Zero-Shot) and our RAP approach. As illustrated in Figures 3, zero-shot generation suffers from high variance. Without a feedback loop, the LLM often generates "plausible-looking" tests that contain logical hallucinations or execution errors (e.g., syntax errors or import failures) that prevent the suite from running.

By utilizing actual execution coverage as the reward, our method provides a "ground-truth" anchor. If a generated test case fails to execute or fails to cover new lines, the MCTS algorithm naturally backtracks and explores alternative branches. This systematic verification ensures that the final test suite is not only textually coherent but functionally effective.

C. Complexity and Computational Budget

The efficiency analysis reveals a "cost of complexity" trend. For simple files, RAP finds high-coverage suites rapidly. However, as code complexity increases (e.g., `graphlib.py`), the search requires more iterations to identify the specific input combinations needed for edge cases.

While our approach is more computationally intensive than a single zero-shot prompt, it is significantly more reliable. The bimodal failure of Pynguin (either 100% or 0%) suggests that traditional tools are brittle in the face of modern Python features. RAP's ability to consistently provide at least partial coverage for difficult files indicates a more robust "floor" for automated testing performance.

D. Limitations and Future Work

While the current framework demonstrates the potential of RAP for test generation, several avenues remain for future exploration to improve performance and generalizability.

- **Model Scaling and Hardware Optimization:** Due to current computational constraints and limited local GPU resources, our experiments were restricted to 7B-parameter models. Future work should evaluate the performance of larger, more powerful LLMs (e.g., Qwen-72B or GPT-4o). Increased model capacity may lead to better reasoning regarding complex state spaces and fewer hallucinations in deep logic paths.
- **Advanced Prompt Engineering and Semantic Guidance:** Future iterations could

implement more granular prompt optimization techniques. Specifically, providing the LLM with data-flow information—such as which variables influence a particular uncovered branch—could significantly improve the "hit rate" of the generated tests. We aim to find more efficient ways to steer the model toward specific edge cases by incorporating static analysis results directly into the prompt context.

- **Cross-Language Generalizability:** While this work focused on Python, the RAP framework is theoretically language-agnostic. Implementing our approach for statically typed languages like Java or C++, or other dynamic languages like JavaScript, would allow us to assess whether the semantic benefits of LLM-guided search are consistent across different programming paradigms.
- **Hierarchical Planning via Nested RAP:** We propose a "Nested RAP" architecture to address highly complex codebases. In this multi-layered approach, an *outer* MCTS layer would focus on branch selection, while an *inner* RAP-MCTS loop would be responsible for generating the specific inputs and assertions required to cover that branch. While this hierarchical structure could significantly improve coverage of deep code paths, further research is required to mitigate the increased computational latency and runtime costs associated with nested search loops.

VI. CONCLUSION

In this work, we introduced RAPiT - a novel test generation framework based on Reasoning via Planning (RAP). By embedding a LLM within a Monte Carlo Tree Search and using real-time execution feedback, we bridged the gap between semantic understanding and systematic code exploration. Our results show that RAP outperforms state-of-the-art SBST tools and zero-shot LLM baselines, achieving higher coverage and

greater stability across both simple and complex Python modules.

REFERENCES

- [1] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*, 3rd. Wiley Publishing, 2011, ISBN: 1118031962.
- [2] S. Lukasczyk and G. Fraser, "Pynguin: Automated unit test generation for python," in *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, ser. ICSE '22, New York, NY, USA: Association for Computing Machinery, Oct. 19, 2022, pp. 168–172, ISBN: 978-1-4503-9223-5. DOI: 10.1145/3510454.3516829.
- [3] S. Hao, Y. Gu, H. Ma, *et al.*, *Reasoning with language model is planning with world model*, Oct. 23, 2023. DOI: 10.48550/arXiv.2305.14992.
- [4] J. Wei, X. Wang, D. Schuurmans, *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.
- [5] A. Prasad, E. Stengel-Eskin, J. C.-Y. Chen, Z. Khan, and M. Bansal, "Learning to generate unit tests for automated debugging," *arXiv preprint arXiv:2502.01619*, 2025.
- [6] Z. Wang, K. Liu, G. Li, and Z. Jin, "Hits: High-coverage llm-based unit test generation via method slicing," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 1258–1268.
- [7] J. Li, G. Li, Y. Li, and Z. Jin, "Structured chain-of-thought prompting for code generation," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 2, pp. 1–23, 2025.
- [8] S. Hao, Y. Gu, H. Luo, *et al.*, *LLM Reasoners: New Evaluation, Library, and Analysis of Step-by-Step Reasoning with Large Language Models*, Aug. 2024. DOI: 10.48550/arXiv.2404.05221. [Online]. Available:

<http://arxiv.org/abs/2404.05221> (visited on 12/18/2025).

- [9] B. Hui, J. Yang, Z. Cui, *et al.*, *Qwen2.5-coder technical report*, Nov. 12, 2024. DOI: 10.48550/arXiv.2409.12186.